

CS641 Course Project

under Prof. Somitra Kumar Sanadhya

Oblivious Ciphertext Compression via Linear Codes

by Pascal Giorgi, Bruno Grenet and Mark Simkin

Presented by

Dhruv Gupta - BT/CSE/220361

Pragati Agrawal - BT/CSE/220779



Department of Computer Science and Engineering
Indian Institute of Technology Kanpur

May 2, 2026

Introduction





- Consider a vector $v \in \mathbb{F}_q^n$ with atmost t non-zero entries, where $t \ll n$.



- ▶ Consider a vector $v \in \mathbb{F}_q^n$ with at most t non-zero entries, where $t \ll n$.
- ▶ We can represent v in two modes —



- ▶ Consider a vector $v \in \mathbb{F}_q^n$ with at most t non-zero entries, where $t \ll n$.
- ▶ We can represent v in two modes —
- ▶ **Dense:** Contains all n entries including zeros. Useful for computations (like multiplication with a matrix). This is the *de-compressed* form.



- ▶ Consider a vector $v \in \mathbb{F}_q^n$ with at most t non-zero entries, where $t \ll n$.
- ▶ We can represent v in two modes —
- ▶ **Dense:** Contains all n entries including zeros. Useful for computations (like multiplication with a matrix). This is the *de-compressed* form.
- ▶ **Sparse:** Contains only the non-zero entries with their indices. Useful for storage, and faster transmission over network. This is the *compressed* form.



- ▶ Consider a vector $v \in \mathbb{F}_q^n$ with at most t non-zero entries, where $t \ll n$.
- ▶ We can represent v in two modes —
- ▶ **Dense:** Contains all n entries including zeros. Useful for computations (like multiplication with a matrix). This is the *de-compressed* form.
- ▶ **Sparse:** Contains only the non-zero entries with their indices. Useful for storage, and faster transmission over network. This is the *compressed* form.
- ▶ (Dense) $[0,0,5,0,7,0,0,0] \equiv$ (Sparse) $[(2,5),(4,7)]$



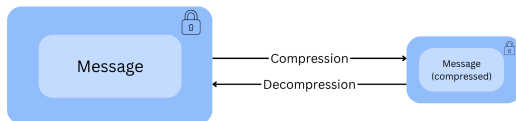
- ▶ Consider a vector $v \in \mathbb{F}_q^n$ with at most t non-zero entries, where $t \ll n$.
- ▶ We can represent v in two modes —
- ▶ **Dense:** Contains all n entries including zeros. Useful for computations (like multiplication with a matrix). This is the *de-compressed* form.
- ▶ **Sparse:** Contains only the non-zero entries with their indices. Useful for storage, and faster transmission over network. This is the *compressed* form.
- ▶ (Dense) $[0,0,5,0,7,0,0,0] \equiv$ (Sparse) $[(2,5),(4,7)]$
- ▶ Converting dense form to sparse form and vice-versa is easy if both are in plaintext.

Introduction



Consider a typical client-server interaction. We want the server to compress/decompress the client's data obliviously.

- ▶ **Oblivious Ciphertext Compression:** Given a dense (element-wise) encrypted vector. Compress it to the encrypted sparse form.
- ▶ **Oblivious Ciphertext Decompression:** Given an encryption for a sparse vector. Decompress it to an encrypted dense vector.
- ▶ Both of these must be done without needing to decrypt, i.e., without getting any information about the underlying plaintext vector.



Homomorphic Encryption (HE)



Homomorphic Encryption (HE)



- ▶ Has 4 algorithms:

Homomorphic Encryption (HE)



► Has 4 algorithms:

- $Gen(1^\lambda)$: generates encryption key ek , decryption key dk .

Homomorphic Encryption (HE)



- ▶ Has 4 algorithms:
 - $Gen(1^\lambda)$: generates encryption key ek , decryption key dk .
 - $Enc(ek, m)$: encrypts m using ek .

Homomorphic Encryption (HE)



► Has 4 algorithms:

- $Gen(1^\lambda)$: generates encryption key ek , decryption key dk .
- $Enc(ek, m)$: encrypts m using ek .
- $Eval(ek, f, c)$: converts a ciphertext vector $c \in C^n$ to $c' \in C'$ s.t. if decrypting c gives $m \in P^n$, then decrypting c' gives $f(m) \in P^m$.

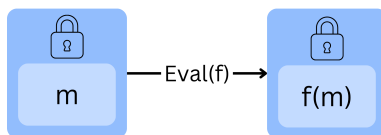
Homomorphic Encryption (HE)



► Has 4 algorithms:

- $Gen(1^\lambda)$: generates encryption key ek , decryption key dk .
- $Enc(ek, m)$: encrypts m using ek .
- $Eval(ek, f, c)$: converts a ciphertext vector $c \in C^n$ to $c' \in C'$ s.t. if decrypting c gives $m \in P^n$, then decrypting c' gives $f(m) \in P^m$.

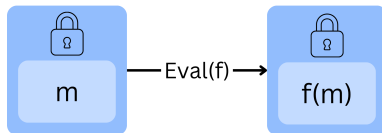
► It allows us to perform certain operations f on ciphertexts directly, without needing decryption.



Homomorphic Encryption (HE)



- ▶ Has 4 algorithms:
 - $Gen(1^\lambda)$: generates encryption key ek , decryption key dk .
 - $Enc(ek, m)$: encrypts m using ek .
 - $Eval(ek, f, c)$: converts a ciphertext vector $c \in C^n$ to $c' \in C'$ s.t. if decrypting c gives $m \in P^n$, then decrypting c' gives $f(m) \in P^m$.
 - $Dec(dk, c)$: decrypts c to m if c is a valid ciphertext, $\perp$ otherwise.
- ▶ It allows us to perform certain operations f on ciphertexts directly, without needing decryption.





► Correctness of HE:

$$\Pr \left[\begin{array}{l} (ek, dk) \leftarrow \text{Gen}(1^\lambda) \\ \mathbf{ct}_0 \leftarrow \text{Enc}(ek, \mathbf{m}) \\ \mathbf{ct}_i \leftarrow \text{Eval}(ek, C_i, \mathbf{ct}_{i-1}) \quad \forall i \in [\ell] \\ \mathbf{m}' \leftarrow \text{Dec}(dk, \mathbf{ct}_\ell) \end{array} : C_\ell(\dots C_1(\mathbf{m}) \dots) \neq \mathbf{m}' \right] \leq \text{negl}(\lambda),$$

► IND-CPA:

$$\Pr \left[\begin{array}{l} (ek, dk) \leftarrow \text{Gen}(1^\lambda) \\ (m_0, m_1) \leftarrow \mathcal{A}(ek) \\ b \leftarrow \{0, 1\} \\ \mathbf{ct} \leftarrow \text{Enc}(ek, m_b) \\ b' \leftarrow \mathcal{A}(\mathbf{ct}) \end{array} : b = b' \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Error Correction Codes





- ▶ Take a vector $m \in P^k$ and convert it to $m' \in P^n$ (code-word) where $n > k$.



- ▶ Take a vector $m \in P^k$ and convert it to $m' \in P^n$ (code-word) where $n > k$.
- ▶ This will add $n - k$ redundant entries which can be used to recover m in case at-most t entries in m are lost/corrupted.





- A linear $[n, k, d]$ code over $\mathbb{F}_q$ is an injective linear map $\mathcal{C} : \mathbb{F}_q^k \rightarrow \mathbb{F}_q^n$. The parameters are:
- k : *message length*,
 - n : *codeword length* ($n > k$),
 - d : *minimum distance* — smallest number of non-zero entries in any non-zero codeword. Also the minimum number of distinct entries between any two codewords. We can prove that these are equivalent. *It determines the error correcting strength of the code.*



- A linear $[n, k, d]$ code over $\mathbb{F}_q$ is an injective linear map $\mathcal{C} : \mathbb{F}_q^k \rightarrow \mathbb{F}_q^n$. The parameters are:
- k : *message length*,
 - n : *codeword length* ($n > k$),
 - d : *minimum distance* — smallest number of non-zero entries in any non-zero codeword. Also the minimum number of distinct entries between any two codewords. We can prove that these are equivalent. *It determines the error correcting strength of the code.*

Hamming weight

Hamming weight of a vector $\mathbf{c}$, $\mathbf{hw}(\mathbf{c}) =$ number of non-zero entries in $\mathbf{c}$.



- ▶ A **generator matrix** $G \in \mathbb{F}_q^{k \times n}$ encodes messages: $\mathbf{c} = \mathbf{m}^T G$.
- ▶ A **parity-check matrix** $H \in \mathbb{F}_q^{(n-k) \times n}$ characterizes codewords: $\mathcal{C} = \{\mathbf{c} \in \mathbb{F}_q^n \mid H\mathbf{c} = \mathbf{0}\}$.

Syndrome

For a parity-check matrix H and any vector $\mathbf{y} = \mathbf{c} + \mathbf{e}$ (codeword $\mathbf{c}$ plus error $\mathbf{e}$), the *syndrome* is:

$$\mathbf{s} = H\mathbf{y} = H(\mathbf{c} + \mathbf{e}) = H\mathbf{e}$$

since $H\mathbf{c} = \mathbf{0}$. The syndrome depends only on the error $\mathbf{e}$, not on the codeword $\mathbf{c}$. Syndrome serves as the error correction code (ECC) for linear codes.



Dual Code

The dual code $\mathcal{C}^\perp$ is the set of all vectors orthogonal to every codeword in $\mathcal{C}$:

$$\mathcal{C}^\perp = \left\{ \mathbf{w} \in \mathbb{F}_q^n \mid \mathbf{w} \cdot \mathbf{c}^T = 0, \forall \mathbf{c} \in \mathcal{C} \right\}.$$

- ▶ **Dimension:** If $\mathcal{C}$ is an $[n, k, d]$ code, then $\mathcal{C}^\perp$ is $[n, n - k, d^\perp]$ code.
- ▶ The parity-check matrix H of $\mathcal{C}$ acts as the generator matrix for $\mathcal{C}^\perp$.



- ▶ We will assume that the codeword is a vector of all zeros, i.e., $\mathbf{c} = \mathbf{0}$.
- ▶ We will treat the message that we want to compress as the error ($\mathbf{e}$).
- ▶ Oblivious Compression simply becomes Oblivious Syndrome Computation, which can be done by homomorphically multiplying the H matrix.
- ▶ Decoding the message simply becomes the error recovery from syndrome.

Core Idea: Decompression as Erasure Correction



- ▶ Decompression is essentially erasure correction in the dual code.
- ▶ Desired values at targeted indices are treated as the only **known symbols** of a codeword.
- ▶ All other $n - t$ indices in the dense vector are viewed as **erasures** (missing data).
- ▶ We want to create a short "seed" m which when de-compressed gives the required dense vector.
- ▶ This is equivalent to finding a message that, when encoded, matches the known symbols.



Let $n, t \in \mathbb{N}$, $\mathcal{M}$ (message space) be a set containing zero, $\mathcal{P}_t = \{v \in \mathcal{M}^n | \mathbf{hw}(v) \leq t\}$. Let $E = (Gen, Enc, Eval, Dec)$ be a HE scheme, δ be the compression rate.

- Oblivious Ciphertext Compression $\equiv$ (OblivCompress, Decompress) such that for any $m \in \mathcal{P}_t$ and any λ –

$$\Pr \left[\begin{array}{l} \text{sparse}(m) = m' \wedge |\mathbf{ct}'| \leq \delta \cdot \xi \cdot n : \\ \begin{array}{l} (ek, dk) \leftarrow \text{Gen}(1^\lambda) \\ \mathbf{ct} \leftarrow \text{Enc}(ek, m) \\ \mathbf{ct}' \leftarrow \text{OblivCompress}(ek, \mathbf{ct}) \\ m' \leftarrow \text{Decompress}(dk, \mathbf{ct}') \end{array} \end{array} \right] \geq 1 - \epsilon,$$



OblivCompress(ek, **ct**)

1 : $f(c) := H \cdot c$

2 : $\mathbf{ct}' \leftarrow \text{E.Eval}(\text{ek}, f, \mathbf{ct})$

3 : **return** $\mathbf{ct}'$

Decompress(dk, **ct**)

1 : $s \leftarrow \text{E.Dec}(\text{dk}, \mathbf{ct})$

2 : $m \leftarrow \mathcal{C}.\text{Decode}(s)$

3 : **return** m



Let $n, t \in \mathbb{N}$, $\mathcal{M}$ (message space) be a set containing zero, $\mathcal{P}_t = \{v \in \mathcal{M}^n | \mathbf{hw}(v) \leq t\}$. Let $E = (\text{Gen}, \text{Enc}, \text{Eval}, \text{Dec})$ be a HE scheme, δ be the compression rate.

- Oblivious Ciphertext Compression with auxilliary input $\equiv$ ($\text{OblivCompressAux}, \text{DecompressAux}$): OblivCompressAux is same as OblivCompress , but now DecompressAux can use a support vector $I = \{i \in [n] | m_i \neq 0\}$ –

$$\Pr \left[\begin{array}{l} \text{sparse}(\mathbf{m}) = \mathbf{m}' \wedge |\mathbf{ct}'| \leq \delta \cdot \xi \cdot n : \\ \begin{array}{l} (\text{ek}, \text{dk}) \leftarrow \text{Gen}(1^\lambda) \\ \mathbf{ct} \leftarrow \text{Enc}(\text{ek}, \mathbf{m}) \\ \mathbf{ct}' \leftarrow \text{OblivCompressAux}(\text{ek}, \mathbf{ct}) \\ \mathbf{m}' \leftarrow \text{DecompressAux}(\text{dk}, \mathbf{ct}', I) \end{array} \end{array} \right] \geq 1 - \epsilon,$$



OblivCompress(ek, **ct**)

1 : $f(c) := H \cdot c$

2 : $\mathbf{ct}' \leftarrow \text{E.Eval}(\text{ek}, f, \mathbf{ct})$

3 : **return** $\mathbf{ct}'$

Decompress(dk, **ct**, I)

1 : $s \leftarrow \text{E.Dec}(\text{dk}, \mathbf{ct})$

2 : $m \leftarrow \mathcal{C}.\text{Decode}(s, I)$

3 : **return** m



Let $n, t \in \mathbb{N}$, $\mathcal{M}$ (message space) be a set containing zero, $\mathcal{P}_t = \{v \in \mathcal{M}^n | \mathbf{hw}(v) \leq t\}$. Let $E = (Gen, Enc, Eval, Dec)$ be a HE scheme, δ be the compression rate.

- Oblivious Ciphertext Decompression $\equiv$ (Compress, OblivDecompress) such that for any $m \in \mathcal{P}_t$ and any $\lambda -$

$$\Pr \left[\begin{array}{l} (m_i = 0 \vee m_i = m'_i) \forall i \in [n] \wedge \\ |\mathbf{ct}| \leq \delta \cdot \xi \cdot n \end{array} : \begin{array}{l} (\mathbf{ek}, \mathbf{dk}) \leftarrow \text{Gen}(1^\lambda) \\ \mathbf{ct} \leftarrow \text{Compress}(\mathbf{ek}, \text{sparse}(\mathbf{m})) \\ \mathbf{ct}' \leftarrow \text{OblivDecompress}(\mathbf{ek}, \mathbf{ct}) \\ \mathbf{m}' \leftarrow \text{Dec}(\mathbf{dk}, \mathbf{ct}') \end{array} \right] \geq 1 - \epsilon,$$



Let $n, t \in \mathbb{N}$, $\mathcal{M}$ (message space) be a set containing zero, $\mathcal{P}_t = \{v \in \mathcal{M}^n | \mathbf{hw}(v) \leq t\}$. Let $E = (Gen, Enc, Eval, Dec)$ be a HE scheme, δ be the compression rate.

- Oblivious Ciphertext Decompression $\equiv$ (Compress, OblivDecompress) such that for any $m \in \mathcal{P}_t$ and any $\lambda -$

$$\Pr \left[\begin{array}{l} (m_i = 0 \vee m_i = m'_i) \forall i \in [n] \wedge \\ |\mathbf{ct}| \leq \delta \cdot \xi \cdot n \end{array} : \begin{array}{l} (\mathbf{ek}, \mathbf{dk}) \leftarrow \text{Gen}(1^\lambda) \\ \mathbf{ct} \leftarrow \text{Compress}(\mathbf{ek}, \text{sparse}(\mathbf{m})) \\ \mathbf{ct}' \leftarrow \text{OblivDecompress}(\mathbf{ek}, \mathbf{ct}) \\ \mathbf{m}' \leftarrow \text{Dec}(\mathbf{dk}, \mathbf{ct}') \end{array} \right] \geq 1 - \epsilon,$$

- Note that we only need m' to match m at non-zero entries of m .



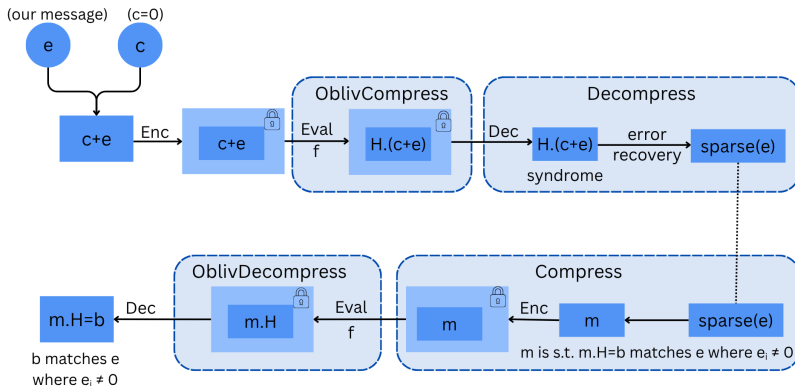
Compress(ek, p)

```
1:  parse  $p$  as  $\{(p_i, i) \mid p_i \neq 0\}$ 
2:   $I := \{i \in [n] \mid p_i \neq 0\}$ 
3:   $m = \mathcal{C}.\text{Solve}(p, I)$ 
4:   $\mathbf{ct} \leftarrow \mathbf{E}.\text{Enc}(\mathbf{ek}, m)$ 
5:  return  $\mathbf{ct}$ 
```

OblivDecompress(ek, $\mathbf{ct}$)

```
1:   $f(u) := u \cdot H$ 
2:   $\mathbf{ct}' \leftarrow \mathbf{E}.\text{Eval}(\mathbf{ek}, f, \mathbf{ct})$ 
3:  return  $\mathbf{ct}'$ 
```

The Algorithm



Example: Oblivious Message Retrieval (OMR)



Setup: Server has access to a mailbox, which has encrypted (message, id) pairs. Alice (Client) wants to obtain her messages anonymously.

1. Alice sends her encrypted id to server.
2. Server matches the client id with the mailbox entries using *homomorphic filtering* and creates a dense vector of the form $[...Enc(0)..., Enc(m_i), ...Enc(0)..., Enc(m_j), ...]$.
3. Server compresses this vector via **Oblivious Compression** scheme and sends it to the client.
4. Client decrypts the received message and decodes it to get its messages.

Example: Batch Private Information Retrieval (PIR)



Setup: Server holds a database of n entries. Bob (Client) wants to retrieve t specific entries anonymously.

1. Bob generates a succinct encrypted "query seed".
2. Bob sends this tiny encrypted seed to the server, hiding the specific locations of the requested items.
3. Server decompresses this seed via **Oblivious Decompression** scheme to create a full-sized dense encrypted selection vector of length n .
4. This dense vector acts as a "shopping list" where Bob's targeted entries are active and all other entries are masked.
Eg., $\text{Enc}([0,0,1,1,0,0,0,1])$
5. Server uses this to get the the required entries from the database homomorphically, **compresses** them and send back to Bob.

Generalized Reed-Solomon Codes (GRS)





Formal Definition:

Let $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q^\times$ be pairwise distinct. The normalized GRS code with parameters $[n, k, n-k+1]$ encodes $\mathbf{m} \in \mathbb{F}_q^k$ as:

$$\mathcal{C}(\mathbf{m}) = (v_1 m(\alpha_1), v_2 m(\alpha_2), \dots, v_n m(\alpha_n)),$$

where $m(x) = \sum_{i=1}^k m_i x^{i-1}$ (a polynomial of degree $< k$) and $v_j = \prod_{i \neq j} \frac{1}{\alpha_j - \alpha_i}$.

Idea:

We create a degree $< k$ polynomial using the message vector $\mathbf{m}$'s entries as coefficients. Evaluate it at n publicly known points. This is used to create the code.

Generalized Reed-Solomon Codes (GRS)



Its parity-check matrix is the Vandermonde matrix:

$$H = \begin{pmatrix} 1 & 1 & \cdots & 1 \\ \alpha_1 & \alpha_2 & \cdots & \alpha_n \\ \alpha_1^2 & \alpha_2^2 & \cdots & \alpha_n^2 \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_1^{n-k-1} & \alpha_2^{n-k-1} & \cdots & \alpha_n^{n-k-1} \end{pmatrix} \in \mathbb{F}_q^{(n-k) \times n}.$$

Why are we using GRS Codes?



- ▶ **Optimal Size (MDS Property):** GRS codes are *Maximum Distance Separable*. They achieve the theoretical limit for compression, requiring exactly $2t$ ciphertexts to recover t non-zero entries.
- ▶ **Strict Linearity:** Compression is reduced to a matrix-vector product. Since this requires only additions and scalar multiplications, it is perfectly compatible with *Additively Homomorphic Encryption*.
- ▶ **Perfect Correctness:** GRS-based compression is deterministic with perfect error recovery.

Why are we using GRS Codes?



- ▶ **Quasi-Linear Efficiency:** The client's decoding time depends on the number of non-zeros (t), instead of all entries (n). This provides an exponential speed-up ($O(t \text{ polylog } t)$) compared to classical $O(n)$ methods.
- ▶ **Syndrome-Only Decoding:** GRS codes allow the client to recover the data directly from the syndrome without ever needing to process the original n -dimensional vector.

- Better time complexity than state of the art algorithms – $\mathcal{O}(n \log t)$, which is nearly linear since typically $t \ll n$.

	Encoding Size	Time		Without Support	Perfect Correctness	Plaintext Space
		Compression	Decompression			
[CDG ⁺ 21]	$\mathcal{O}(t \cdot \kappa)$	$\mathcal{O}(n \cdot \kappa)$	$\mathcal{O}(t \cdot \kappa)$	✓	✗	$\mathbb{G}$
[LT22]	$\mathcal{O}(t \cdot (\lg^2 t + \lg \kappa))$	$\mathcal{O}(n \cdot t)$	$\mathcal{O}(t^3)$	✓	✗	$\mathbb{F}_q$ with $q > 2^\kappa$
[FLS23]	$\mathcal{O}(\kappa \cdot t / \lg t)$	$\mathcal{O}(n \cdot \kappa / \lg t)$	$\mathcal{O}(t \cdot \kappa / \lg t)$	✓	✗	$\mathbb{F}_q$ with $q > 2^\kappa$
[FLS23]	$2t$	$\mathcal{O}(n \lg n)$	$\mathcal{O}(t \cdot \sqrt{n})$	✓	✓	$(\dagger) \mathbb{F}_q$
[FLOS24]	$\mathcal{O}(t + \kappa \cdot \lg \kappa)$	$\mathcal{O}(n \cdot \kappa)$	$\mathcal{O}(t \cdot \kappa)$	✓	✗	$\mathbb{F}_q$ with $q > 2^\kappa$
[BPSY23]	$t \cdot (1 + \epsilon)$	$\mathcal{O}(n \cdot \kappa \cdot 1/\epsilon)$	$\mathcal{O}(t \cdot \kappa \cdot 1/\epsilon)$	✗	✓	$\mathbb{F}_q$
This Work	$2t$	$\mathcal{O}(n \cdot \lg^2 t)$	$\mathcal{O}(t \cdot \lg^2 t \cdot \lg q)$	✓	✓	$(\dagger) \mathbb{F}_q$
This Work	$2t$	$\mathcal{O}(n \cdot \lg t)$	$\mathcal{O}(t \cdot \lg^2 t \cdot \ell)$	✓	✓	$\mathbb{F}_{2^\ell}$
This Work	t	$\mathcal{O}(n \cdot \lg t)$	$\mathcal{O}(t \cdot \lg^2 t)$	✗	✓	$(\dagger) \mathbb{F}_q$
This Work	t	$\mathcal{O}(n \cdot \lg t)$	$\mathcal{O}(t \cdot \lg^2 t)$	✗	✓	$\mathbb{F}_{2^\ell}$



- ▶ **Perfect Correctness:** Unlike prior probabilistic methods (e.g., Cuckoo hashing), this GRS-based approach is fully deterministic. There is zero probability of failure in recovering the non-zero entries.
- ▶ **Theoretical Space Optimality:** It achieves the Singleton Bound. It compresses data into exactly $2t$ ciphertexts, reaching the information-theoretic limit for linear code approaches.
- ▶ **AHE Compatibility:** It relies exclusively on linear operations (additions and scalar multiplications). Fully Homomorphic Encryption (FHE) is not needed. This avoids computational overhead for ciphertext-ciphertext multiplications



- ▶ This oblivious compression scheme is not optimal for encryption schemes with large plaintext spaces, i.e., when $\log \mathbb{F} \gg \log n$.
 - The GRS-based construction uses a syndrome of size $2t \log q$ bits. However, representing t non-zero entries in a vector of length n over $\mathbb{F}_q$ only requires approximately $t(\log q + \log n)$ bits.
- ▶ The paper uses LCH basis to achieve $\mathcal{O}(n \log t)$ for $\mathbb{F}_{2^t}$ fields. Can we find a basis which allows extension to any $\mathbb{F}_q$ with $\mathcal{O}(n \log t)$ time complexity?



- ▶ AHE schemes are malleable and hence vulnerable to certain CCA attacks.
- ▶ Further, any application in practice which uses this compression/decompression scheme, may also need some other functions to be supported by the homomorphic scheme. For eg, Oblivious message retrieval needs homomorphic filtering. This may make the HE scheme more vulnerable.
- ▶ Can we have a compression/decompression scheme which is deterministic, optimal, but does not rely on Homomorphic Encryption?



► **Primary Paper for this Project:**

Pascal Giorgi, Bruno Grenet, and Mark Simkin. *Oblivious Ciphertext Compression via Linear Codes*. Eurocrypt 2026.
Link: eprint.iacr.org/2026/329

► **Reed-Solomon Codes (Video Resource):**

Computerphile (feat. Dr. Mike Pound). *Reed-Solomon Codes*. YouTube, 2015.
Link: youtu.be/6X1OCX-iq9w

► **Coding Theory (Linear Codes & Parity Matrices):**

Ron M. Roth. *Introduction to Coding Theory*. Cambridge University Press, 2006.
Link: doi.org/10.1017/CBO9780511808968



- ▶ **[LT22]** Zeyu Liu and Eran Tromer. *Oblivious Message Retrieval*. CRYPTO 2022.
Link: eprint.iacr.org/2021/1691
- ▶ **[BPSY23]** A. Bienstock, S. Patel, J. Y. Seo, and K. Yeo. *Batch PIR and Labeled PSI with Oblivious Ciphertext Compression*. USENIX Security 2024.
Link: usenix.org/bienstock
- ▶ **Homomorphic Encryption (Foundational):**
R. L. Rivest, L. Adleman, and M. L. Dertouzos. *On data banks and privacy homomorphisms*. Foundations of secure computation, 1978.



► **Security Vulnerabilities in AHE:**

Loftus, J., May, A., Smart, N. P., & Vercauteren, F. (2011).
On CCA-secure somewhat homomorphic encryption. In
International Workshop on Selected Areas in Cryptography
(pp. 55-72). Springer, Berlin, Heidelberg.
Link: doi.org/10.1007/978-3-642-28496-0_4



Thank you!



Extra Slides



- ▶ Developed to optimize arithmetic in binary extension fields $\mathbb{F}_{2^\ell}$.
- ▶ **Construction:** Let $\vec{v} = (v_0, \dots, v_{\ell-1})$ be a basis of $\mathbb{F}_{2^\ell}$ over $\mathbb{F}_2$. Define $\omega_i = \sum_{k=0}^{\ell-1} i_k v_k$, where i_k is the k -th bit of i .
- ▶ **Basis Polynomials:** The LCH basis $\mathbb{X} = \{X_0(x), \dots, X_{2^\ell-1}(x)\}$ is defined as:

$$X_i(x) = \prod_{k=0}^{\ell-1} \left(\prod_{j=0}^{2^k-1} (x - \omega_j) \right)^{i_k}$$

where $\deg(X_i(x)) = i$.



► Key Advantages:

- **Quasi-Linear Arithmetic:** Formal derivatives, products, and divisions are computed in $O(d \log d)$.
- **Additive FFT Compatibility:** Enables fast multi-point evaluation and interpolation over affine subspaces in $O(n \log d)$.
- **Syndrome Recovery:** Allows for error recovery directly from the syndrome in $O(t \log^2 t)$.



Objective: Recover the sparse vector $\mathbf{m}$ from the compressed syndrome $\mathbf{s} = H\mathbf{m}$.

1. **Decryption:** The client decrypts the received ciphertexts to obtain the syndrome $\mathbf{s} \in \mathbb{F}_q^{2t}$.
2. **Error-Locator Polynomial:** Compute the polynomial $\Lambda(x) = \prod_{i \in I} (x - \alpha_i)$ whose roots correspond to the non-zero indices.
 - This is done via **Padé approximation** or the Berlekamp-Massey algorithm.
 - **Complexity:** $\mathcal{O}(M(t) \log t)$, where $M(t)$ is the cost of multiplying two degree $< t$ polynomials.



Objective: Recover the sparse vector $\mathbf{m}$ from the compressed syndrome $\mathbf{s} = H\mathbf{m}$.

3. **Root Finding:** Find the roots of $\Lambda(x)$ in $\mathbb{F}_q$ to identify the set of indices I where $m_i \neq 0$.
 - Uses Berlekamp-Rabin or similar algorithms.
 - **Complexity:** $\mathcal{O}(M(t) \log t \log q)$.
4. **Symbol Recovery:** Once I is known, the client solves for the values m_i using polynomial interpolation or Forney's algorithm.

Result: The client recovers the sparse representation $\text{sparse}(\mathbf{m})$ in quasi-linear time relative to t , not n .



- ▶ **The Scenario:** The client already knows the set of indices I where $m_i \neq 0$ but does not know the actual values.
- ▶ **From Errors to Erasures:**
 - Without support, the client must solve for both **locations** and **values**, requiring $2t$ ciphertexts to achieve minimum distance $d \geq 2t + 1$.
 - With support, the problem reduces to **erasure decoding** — solving only for t unknown values.
- ▶ **Encoding Size Reduction:**
 - The parity-check matrix H only requires t rows instead of $2t$ ($n - k = t$).
 - This results in an **optimal compression rate** of $\delta = t/n$, sending exactly t ciphertexts.



► Algorithmic Changes:

- **Server:** Computes $s = H \cdot ct$ using the t -row matrix.
- **Client:** Skips the complex root-finding and Padé approximation steps.
- **Decoding:** Directly solves the linear system $m_I \cdot H_I = s$ via polynomial interpolation.
- **Efficiency:** Decompression time drops to $\mathcal{O}(M(t) \log t)$.



- ▶ **The Singleton Bound:** For any linear code $[n, k, d]$, there is a fundamental trade-off between redundancy and distance:

$$d \leq n - k + 1$$

- ▶ **Redundancy:** The value $n - k$ represents the number of redundant entries (the syndrome size).
- ▶ **Interpretation:** To increase the distance d (for better error correction), one must either increase the redundancy or decrease the message length.



- ▶ **MDS Definition:** A code is *Maximum Distance Separable* (MDS) if it meets the Singleton Bound exactly: $d = n - k + 1$.
- ▶ **GRS as MDS:** Generalized Reed-Solomon codes are MDS, making them ideal for compression.
- ▶ **Syndrome Size Logic:**
 - **Errors:** To correct t errors, we need $d \geq 2t + 1$. Substituting the MDS property: $(n - k + 1) \geq 2t + 1 \implies \mathbf{n - k \geq 2t}$.
 - **Erasures (Support):** If locations are known, we only need $d \geq t + 1 \implies \mathbf{n - k \geq t}$.
- ▶ **Optimal Compression:** This proves that GRS-based compression sends the **minimum possible number of ciphertexts** (exactly $2t$ or t) needed to recover the data.

Oblivious Decompression: The Solve Primitive



Goal: Given a sparse vector p with non-zero entries at indices I , find a short "seed" vector m that expands into a vector c where $c_i = p_i$ for all $i \in I$.

- ▶ Let H be the parity-check matrix of an $[n, k, d]$ MDS code.
- ▶ We require a vector $m \in \mathbb{F}_q^{n-k}$ such that:

$$m \cdot H_I = p_I$$

where H_I consists of the columns of H at indices I .

- ▶ By the MDS property, every set of $d - 1$ columns in H is linearly independent.
- ▶ Since $|I| \leq t \leq d - 1$, the submatrix H_I is always **full rank**.
- ▶ This ensures the linear system always has at least one solution for m .



► Polynomial Interpolation Perspective:

- For GRS codes, the operation $m \cdot H$ is equivalent to evaluating a polynomial $m(x)$ at points $\alpha_1, \dots, \alpha_n$.
- Finding m such that $m(\alpha_i) = p_i$ for $i \in I$ is exactly the problem of **polynomial interpolation**.

► Erasure Correction Analogy:

- This process can be interpreted as correcting $n - t$ erasures in the dual code $\mathcal{C}^\perp$.
- The compressor "fills in" the missing values to create a valid codeword.

► Complexity:

- The vector m can be computed efficiently in $\mathcal{O}(M(t) \log t)$ finite field operations.